Guia de defensa: Microservicio 1

1. Objetivo del microservicio

El Microservicio 1 simula el control de acceso a un estacionamiento con capacidad limitada. El estacionamiento tiene cuatro espacios y se simula la llegada de diez vehiculos al mismo tiempo.

El problema consiste en controlar el acceso concurrente a un recurso compartido. Los primeros cuatro vehiculos pueden ingresar y los siguientes deben esperar hasta que se libere un espacio.

2. Pseudocodigo original

```
SEMÃ□FORO espacios = 4

PROCEDIMIENTO vehiculo(id)
    escribir("VehÃculo ", id, " llega al estacionamiento")
    wait(espacios)
    escribir("VehÃculo ", id, " ingresa al estacionamiento")
    estacionar(id)
    esperar_tiempo()
    escribir("VehÃculo ", id, " sale del estacionamiento")
    signal(espacios)
FIN PROCEDIMIENTO

INICIO
    PARA i = 1 HASTA 10 HACER
        crear_proceso(vehiculo, i)
    FIN PARA
FIN
```

3. Equivalencia con Java

El codigo principal se encuentra en:

```
backend/microservicios/microservicio-1/src/main/java/edu/ape3/microservicio1/service/ServicioSimulacionEstacionam
```

Crear el semaforo

Pseudocodigo:

```
SEMÃ□FORO espacios = 4
```

Java:

```
Semaphore espacios = new Semaphore(capacidad, true);
```

La simulacion se llama con capacidad `4`, por lo que el semaforo comienza con cuatro permisos. Cada permiso representa un espacio.

Crear los procesos

Pseudocodigo:

```
PARA i = 1 HASTA 10 HACER
    crear_proceso(vehiculo, i)
```

```
FIN PARA
```

Java:

```
ExecutorService ejecutor = Executors.newFixedThreadPool(cantidadVehiculos);

for (int id = 1; id <= cantidadVehiculos; id++) {
    futuros.add(ejecutor.submit(new TareaVehiculo(
            id, espacios, vehiculosDentro, maximoDentro, eventos)));
}
```

`ExecutorService` administra los hilos y cada `TareaVehiculo` representa la ejecucion independiente de un vehiculo.

Procedimiento del vehiculo

El procedimiento del pseudocodigo esta representado por:

```
public ResultadoVehiculo call() throws InterruptedException
```

Dentro del metodo se ejecutan los siguientes pasos:

```
int ocupados = vehiculosDentro.incrementAndGet();
Thread.sleep(TIEMPO_ESTACIONAMIENTO_MILLIS);
vehiculosDentro.decrementAndGet();
```

Equivalencia de operaciones

Pseudocodigo	Java	Funcion
`wait(espacios)`	`espacios.acquire()`	Solicita un permiso y espera si no hay espacio
`estacionar(id)`	`vehiculosDentro.incrementAndGet()`	Registra el ingreso
`esperar_tiempo()`	`Thread.sleep(200)`	Simula el tiempo estacionado
`signal(espacios)`	`espacios.release()`	Libera un espacio
`crear_proceso`	`ejecutor.submit(...)`	Ejecuta un vehiculo concurrentemente

4. Funcionamiento del semaforo

Al iniciar la simulacion:

```
Permisos disponibles: 4
```

Cuando ingresan cuatro vehiculos:

```
Permisos disponibles: 0
```

El quinto vehiculo ejecuta `acquire()` y queda esperando. Cuando uno de los cuatro vehiculos sale, ejecuta `release()` y el quinto puede continuar.

El semaforo garantiza que nunca haya mas de cuatro vehiculos dentro simultaneamente.

El parametro `true` activa una politica de equidad:

```
new Semaphore(capacidad, true)
```

Esto ayuda a atender los hilos que esperan en un orden justo.

5. Patron Modelo-Vista-Controlador

La aplicacion utiliza MVC junto con una capa de servicio:

```
Cliente o frontend
        |
        v
Controlador
        |
        v
Servicio
        |
        v
Modelo
        |
        v
Respuesta JSON
```

Modelo

Se encuentra en:

```
model/
```

Sus clases son:

- `Vehiculo`: representa un vehiculo del dominio.
- `EstadoEstacionamiento`: representa el estado actual del estacionamiento.
- `ResultadoSimulacion`: contiene el resultado general de la simulacion.
- `ResultadoVehiculo`: contiene el resultado individual de un vehiculo.

Ejemplo:

```
public final class Vehiculo {
    private final int id;
}
```

El modelo representa los datos y conceptos del problema.

Vista

Como el proyecto es una API REST, no utiliza una vista HTML tradicional. La vista es la respuesta JSON que recibe el frontend u otro cliente.

Ejemplo:

```
{
  "capacidad": 4,
  "espaciosOcupados": 1,
  "espaciosDisponibles": 3,
  "vehiculosDentro": [5],
  "vehiculosEsperando": [],
  "operacionExitosa": true
}
```

Spring Boot convierte automaticamente los objetos Java en JSON.

Controlador

Se encuentra en:

```
controller/ControladorEstacionamiento.java
```

Su responsabilidad es recibir peticiones HTTP, obtener parametros, llamar al servicio y devolver respuestas.

Ejemplo:

```
@PostMapping("/api/estacionamiento/llegar/{id}")
public EstadoEstacionamiento llegar(@PathVariable int id) {
    return servicioSimulacion.llegarVehiculo(id);
}
```

El controlador no contiene la logica del semaforo.

6. Por que existe la capa Service

La capa Service se encuentra en:

```
service/ServicioSimulacionEstacionamiento.java
```

Contiene la logica de negocio:

- Controlar el semaforo.
- Registrar vehiculos dentro y en espera.
- Simular llegadas concurrentes.
- Controlar entradas y salidas.
- Reiniciar el estacionamiento.
- Generar resultados y eventos.

La capa Service evita colocar esta logica dentro del controlador. Asi, el controlador se mantiene pequeno y la logica puede reutilizarse o probarse independientemente.

La arquitectura completa es:

```
Modelo + Vista + Controlador + Service Layer
```

La capa Service no reemplaza MVC. Es una capa adicional para separar la logica de negocio de la comunicacion HTTP.

7. Flujo de una simulacion

Cuando el cliente solicita:

```
GET /api/simular
```

ocurre lo siguiente:

1. `ControladorEstacionamiento` recibe la solicitud.
2. El controlador llama a `simular(4, 10)`.
3. El servicio crea un semaforo con cuatro permisos.
4. Se crean diez tareas, una por cada vehiculo.
5. Cada vehiculo solicita un permiso.
6. Solo cuatro vehiculos ingresan simultaneamente.
7. Los demas esperan.
8. Los vehiculos salen despues de 200 milisegundos.
9. Cada salida libera un permiso.
10. Se genera un `ResultadoSimulacion`.
11. Spring devuelve el resultado en formato JSON.

8. Uso de `AtomicInteger`

Varios hilos modifican simultaneamente la cantidad de vehiculos dentro:

```
AtomicInteger vehiculosDentro = new AtomicInteger();
```

Se utiliza `AtomicInteger` porque sus operaciones son seguras en concurrencia. Esto evita que dos hilos sobrescriban accidentalmente el mismo valor.

Tambien se utiliza para calcular el maximo de vehiculos dentro:

```
maximoDentro.accumulateAndGet(ocupados, Math::max);
```

9. Uso de `try-finally`

La salida se implementa de esta forma:

```
try {
    Thread.sleep(TIEMPO_ESTACIONAMIENTO_MILLIS);
    return new ResultadoVehiculo(id, milisegundosEsperados, true);
} finally {
    vehiculosDentro.decrementAndGet();
    espacios.release();
}
```

El bloque `finally` garantiza que el espacio se libere aunque ocurra un error o se interrumpa el hilo. Sin este bloque, el estacionamiento podria quedar bloqueado.

10. Endpoints disponibles

```
GET  /health
GET  /api
GET  /api/simular
GET  /api/estacionamiento/estado
POST /api/estacionamiento/llegar/{id}
POST /api/estacionamiento/salir/{id}
POST /api/estacionamiento/reiniciar
```

11. Preguntas frecuentes para la defensa

Â¿Por que solo pueden ingresar cuatro vehiculos?

Porque el semaforo se crea con cuatro permisos. Cada vehiculo consume un permiso al entrar y lo devuelve al salir.

Â¿Que ocurre con el quinto vehiculo?

Queda bloqueado en `espacios.acquire()` hasta que otro vehiculo ejecute `espacios.release()`.

Â¿Que representa `acquire()`?

Representa la operacion `wait` del pseudocodigo.

Â¿Que representa `release()`?

Representa la operacion `signal` del pseudocodigo.

Â¿Por que el controlador no tiene toda la logica?

Porque el controlador debe encargarse de HTTP. La logica de negocio se ubica en el Service para mantener separadas las responsabilidades.

Â¿Cual es la vista en una API REST?

La vista es la respuesta JSON que Spring genera a partir de los objetos del modelo.

Â¿Que diferencia existe entre `Vehiculo` y `ResultadoVehiculo`?

`Vehiculo` representa la entidad del dominio. `ResultadoVehiculo` representa el resultado de la participacion de ese vehiculo en la simulacion.

Â¿Como se verifica que la solucion funciona?

Al ejecutar la simulacion con capacidad cuatro y diez vehiculos, el valor maximo de vehiculos dentro debe ser cuatro. Esto demuestra que el semaforo controla correctamente el acceso.

12. Defensa corta

Mi microservicio controla el acceso concurrente a un estacionamiento con cuatro espacios. Utilizo un `Semaphore` con cuatro permisos. `acquire()` representa `wait` y `release()` representa `signal`. Cada vehiculo se ejecuta como una tarea independiente mediante `ExecutorService`. Los primeros cuatro vehiculos ingresan y los restantes esperan hasta que se libere un espacio. El controlador recibe las peticiones HTTP, el Service contiene la logica de concurrencia y el modelo representa los vehiculos y los resultados. Como es una API REST, la vista corresponde a las respuestas JSON generadas por Spring Boot.